# 🧠 DecodeLabs AI Engineering Internship — Week 2

## Project 2: Data Classification Using AI

> **Internship Track:** AI Engineering Intern
> **Framework:** Scikit-Learn | IPO Pipeline (Input → Process → Output)
> **Dataset:** Iris Benchmark Dataset (UCI / Fisher 1936)
> **Algorithm:** K-Nearest Neighbors (KNN) Classifier
> **Standard Tuning:** K = 5 (Blueprint Compliance)

---

## 📋 Table of Contents



---

## 🎯 Project Overview

This project implements a **complete Machine Learning classification pipeline** using the classic **Iris Dataset** to classify iris flowers into three species:

- 🌸 *Iris Setosa*
- 🌼 *Iris Versicolor*
- 🌺 *Iris Virginica*

The model uses **K-Nearest Neighbors (KNN)** algorithm with strict **K=5** tuning as per the DecodeLabs blueprint guidelines. The entire workflow follows the professional **IPO (Input → Process → Output)** architecture.

---

## ✨ Key Features

| Feature | Description |
|---|---|
| ✅ **IPO Pipeline** | Clean separation of Input, Process, and Output phases |
| ✅ **Data Leakage Prevention** | Split-before-scale methodology applied strictly |

Table 1 – continued

| Feature | Description |
|---|---|
| ✅ **Elbow Method** | Visual evaluation of optimal K (1-30) with blueprint K=5 highlighted |
| ✅ **StandardScaler** | Feature normalization with Gatekeeper Rule |
| ✅ **Stratified Split** | 80/20 train-test split preserving class proportions |
| ✅ **Confusion Matrix** | Detailed heatmap with True/False Positive analysis |
| ✅ **Classification Report** | Precision, Recall, and F1-Score per class |
| ✅ **Dark Theme Visuals** | Professional CLI and plot design for presentation |
| ✅ **Reproducibility** | Fixed `random_state=42` across all operations |

---

## 🛠️ Technical Stack

```
Python 3.x
├── NumPy          → Numerical computations
├── Matplotlib     → Data visualization & plotting
├── Seaborn        → Statistical heatmaps
├── Scikit-Learn   → ML algorithms & metrics
└── Standard Lib   → Warnings management
```

---

## 📁 Project Structure

```
DecodeLabs-Week2-KNN-Iris/
│
├── 📄 README.md                          ← You are here
├── 📄 decodelabs_knn.py                  ← Main Python script
├── 📄 decodelabs_knn_diagnostics.png     ← Output visualization
│
└── 📂 assets/
    ├── 📸 screenshot_code.png            ← Code execution screenshot
    ├── 📸 screenshot_output.png          ← Terminal/Console output
    ├── 📸 screenshot_plot.png            ← Elbow curve & Confusion Matrix
    └── 📄 requirements.txt               ← Dependency list
```

---

## ⚙️ Installation & Setup

### Step 1: Clone or Download the Repository

```
git clone https://github.com/yourusername/DecodeLabs-Week2-KNN-Iris.git
cd DecodeLabs-Week2-KNN-Iris
```

### Step 2: Install Dependencies

```
pip install numpy matplotlib seaborn scikit-learn
```

Or use the requirements file:

```
pip install -r requirements.txt
```

### Step 3: Verify Installation

```
python -c "import sklearn; print('Scikit-Learn version:', sklearn.__version__)"
```

---

## 🚀 How to Run

### Run the Main Script

```
python decodelabs_knn.py
```

### Expected Output

```
======================================================================
  DecodeLabs AI Internship — Project 2: Data Classification Using AI
  KNN Iris Classifier | IPO Pipeline                        Week 2
======================================================================

======================================================================
  INPUT & PROCESS PHASE — Loading, Splitting & Scaling
======================================================================

  Dataset      : Iris Benchmark  (UCI / Fisher 1936)
  Samples      : 150  |  Features : 4
  Target Classes : ['setosa' 'versicolor' 'virginica']

  [Split Success] Training: 120 samples | Test: 30 samples
  [Gatekeeper Rule Applied] StandardScaler → Data Leakage Prevented.
  Post-scale mean (Train Feature 0): 0.00 (Target: 0.0)
  Post-scale var  (Train Feature 0): 1.00 (Target: 1.0)

... (pipeline continues)
```

---

## 🔬 IPO Pipeline Breakdown

### 📥 INPUT PHASE

```
# 1. Load Dataset
iris = load_iris()
X, y = iris.data, iris.target

# 2. Split FIRST (Prevent Data Leakage)
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.20, random_state=42, stratify=y
)

# 3. Scale AFTER Split (Gatekeeper Rule)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # Fit ONLY on train
X_test_scaled  = scaler.transform(X_test)         # Transform test
```

### ⚙️ PROCESS PHASE

```
# a) INSTANTIATE
model = KNeighborsClassifier(n_neighbors=5, metric="euclidean")

# b) FIT
model.fit(X_train_scaled, y_train)

# c) PREDICT
predictions = model.predict(X_test_scaled)
```

### 📤 OUTPUT PHASE

```
# 1. Accuracy Score
accuracy = accuracy_score(y_test, predictions)

# 2. Confusion Matrix
cm = confusion_matrix(y_test, predictions)

# 3. Classification Report (Precision, Recall, F1)
report = classification_report(y_test, predictions, target_names=iris.target_names)
```

---

## 🧠 Critical ML Concepts Applied

### 1 Data Leakage Prevention (The Gatekeeper Rule)

> **Problem:** Scaling before splitting leaks test data statistics (mean/variance) into training.
> **Solution:** `fit_transform()` on Train → `transform()` on Test only.

### 2 Stratified Sampling

> Ensures each class (Setosa, Versicolor, Virginica) maintains its original proportion (33.3% each) in both train and test sets.

### 3 Feature Scaling (Standardization)

> KNN is distance-based. Unscaled features (e.g., petal length in cm vs. sepal width in mm) would dominate. StandardScaler converts all features to mean=0, variance=1.

### 4 Elbow Method vs. Blueprint Compliance

> While the Elbow Method evaluates K=1 to K=30 for visualization, the **final model strictly uses K=5** as per project instructions.

### 5 Reproducibility

> Fixed `random_state=42` ensures identical results every run — critical for debugging and peer review.

---

## 📊 Results & Metrics

### Overall Accuracy

```
Overall Accuracy : ~96.67% - 100.00%
```

*(May vary slightly based on random split, but typically 96-100% on Iris)*

### Confusion Matrix

| | Predicted Setosa | Predicted Versicolor | Predicted Virginica |
|---|---|---|---|
| **Actual Setosa** | 10 | 0 | 0 |
| **Actual Versicolor** | 0 | 10 | 0 |
| **Actual Virginica** | 0 | 1 | 9 |

### Classification Report

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00        10
   virginica       1.00      0.90      0.95        10

    accuracy                           0.97        30
   macro avg       1.00      0.97      0.98        30
weighted avg       1.00      0.97      0.98        30
```

---

## 📈 Visualizations

The script generates a **dual-panel diagnostic plot** saved as `decodelabs_knn_diagnostics.png`:

### Panel 1: Elbow Method Curve

- X-axis: K values (1-30)
- Y-axis: Error Rate
- **Orange dashed line** marks the blueprint standard K=5
- Shows how error stabilizes after K=5

**Panel 2: Confusion Matrix Heatmap**

- Color-coded intensity (YlOrRd colormap)
- Annotated with exact counts
- Dark professional theme matching CLI design

---

## 📸 Screenshots

> *Add your execution screenshots in the* `/assets/` *folder and reference them here:*

| Screenshot | Description |
|---|---|
| | Python script in editor<br>Terminal execution output<br>Generated elbow curve & confusion matrix |

---

## 🎓 Learning Outcomes

Through this project, I have demonstrated:

- ✅ Understanding of **supervised classification** workflows
- ✅ Ability to prevent **data leakage** in preprocessing
- ✅ Proficiency in **Scikit-Learn** API (fit/predict pattern)
- ✅ Knowledge of **distance-based algorithms** and their sensitivity to scale
- ✅ Skill in **model evaluation** using multiple metrics (not just accuracy)
- ✅ Capability to create **publication-ready visualizations**
- ✅ Adherence to **strict project guidelines** and blueprint compliance

---


## 🙏 Acknowledgements

- **DecodeLabs** — For providing this structured internship and learning opportunity
- **Scikit-Learn Team** — For the robust ML framework
- **UCI Machine Learning Repository** — For the classic Iris dataset
- **Ronald Fisher** — Original collector and publisher of the Iris dataset (1936)


---

## 📫 Contact

For any queries regarding this project:


- **Intern Name:** [Your Name]
- **Email:** [your.email@example.com]
- **LinkedIn:** [Your LinkedIn Profile]
- **GitHub:** [Your GitHub Profile]


---

⭐ **DecodeLabs AI Engineering Internship — Week 2** ⭐

*Submitted as part of the official internship program.*